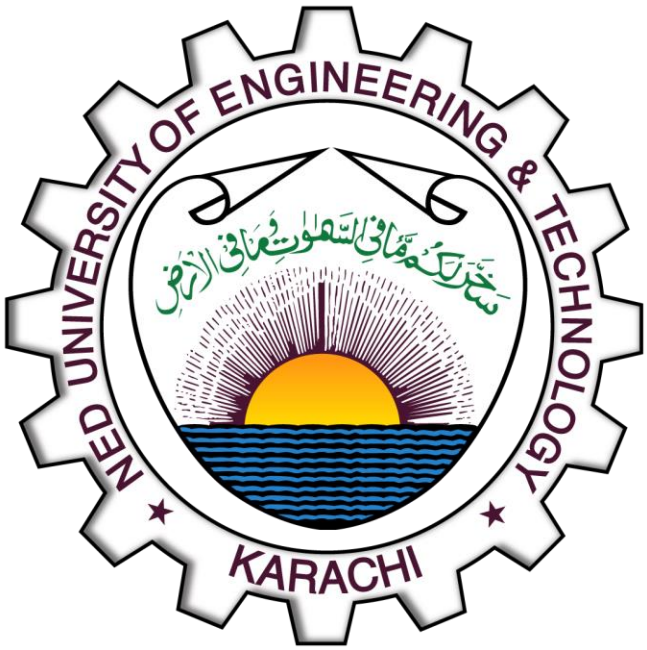
DATA STRUCTURE ALGORITHMS & APPLICATIONS

[CT-159]



NAME: TAHA AHMED MALLICK

ROLL NO: CT-25183

DEPARTMENT: BCIT **BATCH:** 2025

YEAR & SECTION: SSCS-D

LAB : #05

QUESTION#1:

CODE:

```
#include <iostream>
using namespace std;

class MyCircularDequeue
{
    int *data;
    size_t cap, front, rear, size;

public:
    MyCircularDequeue(int k) : cap(k), size(0), front(0), rear(0)
    {
        data = new int[k];
    }
    ~MyCircularDequeue()
    {
        delete[] data;
    }
    bool insertFront(int val)
    {
        if (isFull())
            return false;
        front = (front - 1 + cap) % cap;
        data[front] = val;
        size++;
        return true;
    }
    bool insertLast(int val)
    {
        if (isFull())
            return false;
        data[rear] = val;
        rear = (rear + 1) % cap;
        size++;
        return true;
    }
    bool deleteFront()
    {
        if (isEmpty())
            return false;
        front = (front + 1) % cap;
```

```
        size--;
        return true;
    }
    bool deleteLast()
    {
        if (isEmpty())
            return false;
        rear = (rear - 1 + cap) % cap;
        size--;
        return true;
    }
    int getFront() const
    {
        if (!isEmpty())
            return data[front];
        return -1;
    }
    int getRear() const
    {
        if (!isEmpty())
                return data[(rear - 1 + cap) % cap];
        return -1;
    }
    bool isEmpty() const { return size == 0; }
    bool isFull() const { return size == cap; }
};

string format(bool val)
{
    return val ? "True" : "False";
}

int main()
{
    MyCircularDequeue *deque = new MyCircularDequeue(3);
    cout << format(deque->insertLast(1)) << endl;
    cout << format(deque->insertLast(2)) << endl;
    cout << format(deque->insertFront(3)) << endl;
    cout << format(deque->insertFront(4)) << endl;
    cout << deque->getRear() << endl;
    cout << format(deque->isFull()) << endl;
    cout << format(deque->deleteLast()) << endl;
    cout << format(deque->insertFront(4)) << endl;
    cout << deque->getFront() << endl;
    return 0;}
```

OUTPUT:

```
True
True
True
False
2
True
True
True
4
```

QUESTION#2:

CODE:

```cpp
#include <iostream>
#include <queue>
using namespace std;

int findTheWinnerMath(int n, int k) {
    queue<int> friends;
    for (int i = 1; i <= n; i++)
        friends.push(i);
    while (friends.size() > 1)
    {
        for (int steps = 1; steps < k; steps++) {
            int curr = friends.front();
            friends.pop();
            friends.push(curr);
        }
        friends.pop();
    }
    return friends.front();
}

int main() {
    int n = 5, k = 2;
    cout << "Winner Friend: " << findTheWinnerMath(n, k) << endl;
    return 0;
}
```

OUTPUT:

```
Winner Friend: 3
```

QUESTION#3:

CODE:

```
#include <iostream>
#include <queue>
#include <vector>
#include <cstdlib>
#include <ctime>
using namespace std;

class Call
{
    static size_t callID;
    string name;
    size_t arrivalTime, ID;

public:
    Call(string name, size_t arrivalTime) : name(name), arrivalTime(arrivalTime)
    {
        ID = callID;
        callID++;
    }
    string getName() const { return name; }
    size_t getTime() const { return arrivalTime; }
    size_t getID() const { return ID; }
};

size_t Call::callID = 0;

class CallCenter
{
    queue<Call> calls;
    size_t CSR, time;
    vector<bool> availibility;
    vector<size_t> timeLeft;

public:
    CallCenter(size_t CSR) : time(0), CSR(CSR), availibility(CSR, true),
timeLeft(CSR, 0)
    {
        cout << "\t\t\e[1;33m----- Call Center with " << CSR << " CSRS -----
\n\n\e[0m";
    }
```

```
    void addCall(Call &call)
    {
        calls.push(call);
        cout << "\e[1;32m[Arrival] Call ID: " << call.getID() << " (" <<
call.getName() << ") joined at time: " << call.getTime() << "\e[0m" << endl;
    }

    void processCalls()
    {
        for (int i = 0; i < availibility.size(); i++)
            if (availibility[i] && !calls.empty())
            {
                Call dispatch = calls.front();
                calls.pop();
                availibility[i] = false;
                timeLeft[i] = rand() % 6 + 1; // assigns a random call duration
                cout << "\e[1;34m[Dispatched : CSR#" << i + 1 << "] Call ID: " <<
dispatch.getID() << ", (" << dispatch.getName() << ") for duration: " <<
timeLeft[i] << "\e[0m\n";
            }
    }

    void tick()
    {
        for (int i = 0; i < availibility.size(); i++)
            if (!availibility[i])
            {
                timeLeft[i]--;
                if (timeLeft[i] == 0)
                {
                    availibility[i] = true;
                    cout << "\e[1;35m[Completed] CSR#" << i + 1 << " has
completed there call and now available.\n\e[0m";
                }
            }
        processCalls();
        time++;
    }

    bool hasPendingWork() const
    {
        if (!calls.empty())
            return true;
        for (bool busy : availibility)
```

```
            if (!busy)
                return true;
        return false;
    }

    size_t getTime() const { return time; }
};

int main()
{
    srand(time(0)); // random number gen seed

    CallCenter center(2);
    vector<Call> preplannedCalls = {
        Call("Taha", 0),
        Call("Danial", 2),
        Call("Yousuf", 2),
        Call("Ahmed", 5),
        Call("Yahya", 6)};

    size_t index = 0;

    while (index < preplannedCalls.size() || center.hasPendingWork())
    {
        cout << "\n\e[1m[Time: " << center.getTime() << "]\n\e[0m";
        while (index < preplannedCalls.size() && preplannedCalls[index].getTime()
== center.getTime())
            center.addCall(preplannedCalls[index++]);
        center.tick();
    }
    return 0;
}
```

OUTPUT:

```
                ----- Call Center with 2 CSRS -----


[Time: 0]
[Arrival] Call ID: 0 (Taha) joined at time: 0
[Dispatched : CSR#1] Call ID: 0, (Taha) for duration: 5

[Time: 1]
```

```
[Time: 2]
[Arrival] Call ID: 1 (Danial) joined at time: 2
[Arrival] Call ID: 2 (Yousuf) joined at time: 2
[Dispatched : CSR#2] Call ID: 1, (Danial) for duration: 1

[Time: 3]
[Completed] CSR#2 has completed there call and now available.
[Dispatched : CSR#2] Call ID: 2, (Yousuf) for duration: 5

[Time: 4]

[Time: 5]
[Arrival] Call ID: 3 (Ahmed) joined at time: 5
[Completed] CSR#1 has completed there call and now available.
[Dispatched : CSR#1] Call ID: 3, (Ahmed) for duration: 5

[Time: 6]
[Arrival] Call ID: 4 (Yahya) joined at time: 6

[Time: 7]

[Time: 8]
[Completed] CSR#2 has completed there call and now available.
[Dispatched : CSR#2] Call ID: 4, (Yahya) for duration: 6

[Time: 9]

[Time: 10]
[Completed] CSR#1 has completed there call and now available.

[Time: 11]

[Time: 12]

[Time: 13]

[Time: 14]
[Completed] CSR#2 has completed there call and now available.
```

QUESTION#4:

CODE:

```cpp
#include <iostream>
#include <vector>
using namespace std;

class ProductOfNumbers {
    vector<int> products;

public:
    ProductOfNumbers() : products() { products.push_back(1); }

    void add(int num) {
        if (num == 0) {
            products.clear();
            products.push_back(1);
        } else
            products.push_back(num * products.back());
    }

    int getProduct(int k) {
        if (k >= (int)products.size()) return 0;
        return products.back() / products[products.size() - k - 1];
    }
};

int main() {
    ProductOfNumbers* product = new ProductOfNumbers();
    product->add(3);
    product->add(0);
    product->add(2);
    product->add(5);
    product->add(4);
    cout << product->getProduct(2) << endl;
    cout << product->getProduct(3) << endl;
    cout << product->getProduct(4) << endl;
    product->add(8);
    cout << product->getProduct(2) << endl;
    return 0;
}
```

OUTPUT:

```
20
40
0
32
```

QUESTION#5:

CODE:

```
#include <iostream>
using namespace std;

class DataStream
{
    int val, k, streak;

public:
    DataStream(int val, int k) : val(val), k(k), streak(0) {}

    bool consec(int num)
    {
        if (num == val)
            streak++;
        else
            streak = 0;
        return streak >= k;
    }
};

string format(bool val)
{
    return val ? "True" : "False";
}

int main()
{
    DataStream *dataStream = new DataStream(4, 3);
    cout << format(dataStream->consec(4)) << endl;
    cout << format(dataStream->consec(4)) << endl;
    cout << format(dataStream->consec(4)) << endl;
    cout << format(dataStream->consec(3)) << endl;
    return 0;
}
```

OUTPUT:

```
False
False
True
False
```